

Raport końcowy projektu:

Yahul Clicker

Zespół projektowy: Krystian, Tymoteusz

Technologia: C++, SDL (Simple DirectMedia Layer), SQLite

Architektura systemu: Model-View-Controller (MVC) / Data-Driven Architecture

Tydzień 1 projektu:

Tymoteusz(khhv - mama sie nie liczy):

Integracja silnika bazodanowego: Dokonano implementacji i parametryzacji lekkiego, bezserwerowego systemu zarządzania bazą danych SQLite w strukturę aplikacji.

Projektowanie relacyjnego modelu danych: Opracowano i zaimplementowano schematy tabel bazodanowych za pomocą języka SQL. Utworzono struktury dedykowane dla trzech kluczowych encji:

- `upgrades` – ewidencja stanu i dostępności modyfikatorów systemowych.
- `stats` – przechowywanie danych o zapisie i stanie pieniędzy w grze

Weryfikacja integralności i testy I/O: Przeprowadzono serię testów jednostkowych i integracyjnych weryfikujących poprawność operacji zapisu (INSERT/UPDATE) oraz odczytu (SELECT) danych. Potwierdzono spójność danych oraz brak anomalii w komunikacji z plikiem bazy danych.

Krystian:

Konfiguracja środowiska programistycznego: Skonstruowano struktury projektu w języku C++, zapewniając optymalną konfigurację kompilatora oraz relacji między modułami.

Inicjalizacja podsystemu multimedialnego: Przeprowadzono poprawną integrację oraz inicjalizację biblioteki SDL (Simple DirectMedia Layer), stanowiącej warstwę abstrakcji sprzętowej dla grafiki i dźwięku.

Implementacja głównej pętli systemu (Game Loop): Zaprojektowano i wdrożono deterministyczną pętlę sterującą, odpowiedzialną za cykliczne przetwarzanie zdarzeń, aktualizację stanu obiektów oraz renderowanie klatek obrazu.

Moduł detekcji zdarzeń wejściowych: Opracowano architekturę obsługi urządzeń peryferyjnych, ze szczególnym uwzględnieniem precyzyjnego przechwytywania i interpretacji zdarzeń generowanych przez mysz komputerową.

Tydzień 2:

Tymoteusz(goj):

Projekt i implementacja interfejsu użytkownika (UI): Wykonano implementację minimalistycznego interfejsu graficznego, zawierającego dwa kluczowe komponenty:

- Moduł wizualizacji stanu: Dynamiczne pole tekstowe prezentujące bieżący stan licznika punktów.
- Interaktywny komponent wejściowy: Fizyczny przycisk ekranowy dedykowany do obsługi zdarzeń kliknięcia.

Integracja warstwy UI z systemem bazodanowym: Zaprojektowano i zaimplementowano procedurę cyklicznego zapisu stanu punktowego bezpośrednio do struktur relacyjnych SQLite. Zapewnia to trwałość postępu użytkownika (persystencję danych) pomiędzy kolejnymi sesjami uruchomieniowymi aplikacji.

Krystian:

Implementacja mechaniki interakcji bezpośredniej: Zaprojektowano i wdrożono algorytm detekcji kolizji kursora z obiektami aktywnymi (tzw. *picking/click mechanics*), stanowiący podstawowy mechanizm wejściowy aplikacji.

Moduł agregacji i inkrementacji punktowej: Opracowano deterministyczny licznik punktów, zarządzający stanem punktowym w czasie rzeczywistym i reagujący na zdarzenia wejściowe.

Projekt podsystemu odświeżania ekranu: Wdrożono zaawansowany system aktualizacji bufora ramki (Screen Update System). Architektura ta minimalizuje narzut obliczeniowy poprzez odświeżanie wyłącznie zmodyfikowanych elementów środowiska graficznego.

Tydzień 3:

Tymoteusz(femboy):

Implementacja interfejsu handlowego (Shop UI): Zaprojektowano i zaimplementowano dedykowany moduł graficzny GUI dla sklepu. Odpowiada on za czytelną prezentację dostępnych ulepszeń, ich bieżących kosztów oraz poziomów zaawansowania.

Strukturyzacja tabeli relacyjnej: Dokonano finalnej parametryzacji i optymalizacji tabeli `upgrades` w bazie danych SQLite, przystosowując ją do przechowywania unikalnych identyfikatorów modyfikatorów oraz powiązanych z nimi stanów.

Persystencja zdarzeń zakupowych: Opracowano i wdrożono procedury zapisu transakcji. Każda udana operacja zakupu ulepszenia jest natychmiastowo lub sesyjnie utrwalana w bazie SQLite, co gwarantuje pełną ciągłość stanu systemu po restarcie aplikacji.

Krystian:

Projektowanie systemu modyfikatorów (Upgrade System): Opracowano architekturę ulepszeń, umożliwiającą dynamiczne skalowanie wartości bazowej przyrostu punktowego (inkrementacja wartości generowanej przez pojedyncze zdarzenie wejściowe / kliknięcie).

Algorytm kalkulacji kosztów: Zaimplementowano funkcję matematyczną odpowiedzialną za obliczanie zmiennego kosztu kolejnych poziomów ulepszeń, zapewniając stabilność balansu ekonomicznego systemu.

Logika transakcyjna (Kupowanie): Wdrożono maszynę stanów zarządzającą walidacją operacji zakupowych. System weryfikuje warunek posiadania wymaganych zasobów, dokonuje dedukcji odpowiedniej liczby punktów oraz aktualizuje poziom nabytego modyfikatora.

Tydzień 4:

Tymoteusz(ltn-):

Persystencja stanu struktur generujących: Rozszerzono schemat bazy danych SQLite oraz zaimplementowano procedury zapisu stanu aktywnych generatorów pasywnych. Wdrożone kwerendy SQL gwarantują, że po ponownym uruchomieniu aplikacji system poprawnie odtwarza bazową wydajność sekundową przypisaną do profilu użytkownika.

Krystian:

Implementacja systemu pasywnej generacji (Idle Generation): Zaprojektowano i wdrożono algorytm odpowiedzialny za cykliczne, automatyczne zwiększanie puli punktów w interwałach sekwencyjnych, niezależnie od zdarzeń wejściowych myszy.

Architektura podsystemu timerów: Opracowano dedykowany system zarządzania czasem (Timer System) bazujący na wewnętrznym zegarze procesora lub funkcjach czasu biblioteki SDL. Układ ten zapewnia precyzyjne odliczanie interwałów

jednosekundowych ((1000 ms)) przy minimalnym narzucie na wątek główny aplikacji.

Tydzień 5:

Tymoteusz(pianista od siedmiu boleści):

- Integracja dwuwymiarowych zasobów graficznych: Zastąpiono surowe, testowe elementy interfejsu docelowymi teksturami. Zaimplementowano dedykowane komponenty wizualne dla przycisków oraz statyczne tło aplikacji.
- Implementacja dynamicznych sprzężeń zwrotnych: Zaprojektowano i wdrożono system mikroanimacji reagujący na zdarzenie kliknięcia (Click Animation). Zapewnia on natychmiastową, wizualną reakcję zwrotną dla użytkownika, podnosząc jakość interakcji z programem.
- Optymalizacja i korekta interfejsu (UI English): Przeprowadzono finalny przegląd rozmieszczenia elementów GUI. Skorygowano marginesy, wyrównanie komponentów oraz czytelność czcionek, osiągając pełną spójność wizualną.

Krystian:

- Kalibracja balansu ekonomicznego (Game Balancing): Przeprowadzono analizę numeryczną krzywej progresji. Dostrojono współczynniki kosztów ulepszeń oraz efektywność generatorów, eliminując anomalie w tempie przyrostu zasobów.
- Refaktoryzacja struktury kodu (Code Refactoring): Dokonano reorganizacji kodu źródłowego C++. Wdrożono wzorce projektowe, usunięto redundancje oraz zoptymalizowano zarządzanie pamięcią, co przełożyło się na zwiększenie czytelności i modularności systemu.
- Konsolidacja podsystemu zapisu (Save System): Sfinalizowano architekturę modułu odpowiedzialnego za zrzut stanu aplikacji. Zintegrowano rozproszone mechanizmy zapisu w jednolity, transakcyjny system persystencji danych bazodanowych.

Tydzień 6(7):

Tymoteusz:

Działania obejmowały architekturę przepływu ekranów aplikacji, prezentację danych historycznych oraz końcowe testy integracyjne.

- Implementacja modułu nawigacji (Menu Startowe): Zaprojektowano i wdrożono ekran główny aplikacji. Moduł ten zarządza stanami początkowymi systemu, umożliwiając bezpieczną inicjalizację nowej sesji lub załadowanie istniejącego profilu.
- Projekt panelu analitycznego (Ekran Statystyk): Opracowano dedykowany widok agregujący dane telemetryczne użytkownika. Ekran ten pobiera i dynamicznie prezentuje z bazy SQLite historyczne wskaźniki wydajności oraz sumaryczne osiągnięcia gracza.
- Walidacja i testy końcowe (Testing): Przeprowadzono kompleksowe testy funkcjonalne, regresyjne oraz integracyjne. Zweryfikowano poprawność interakcji na linii UI → Logika → Baza Danych, potwierdzając zgodność produktu ze specyfikacją techniczną.

Krystian:

Prace koncentrowały się na zapewnieniu bezawaryjnego działania aplikacji oraz minimalizacji zużycia zasobów sprzętowych.

- Profilowanie i eliminacja błędów (Bug Fixing): Przeprowadzono szczegółowy proces debugowania. Zidentyfikowano i usunięto błędy logiczne, anomalie w pętli gry oraz wycieki pamięci, stabilizując działanie aplikacji w scenariuszach długoterminowych.
- Optymalizacja wydajnościowa (Optimization): Zoptymalizowano operacje wejścia/wyjścia (I/O) przy komunikacji z SQLite oraz algorytmy renderowania SDL. Zredukowano narzut na procesor (CPU) i czas dostępu do bazy danych, uzyskując stabilną liczbę klatek na sekundę (FPS).

Podsumowanie pracy:

1. Architektura i Ewolucja Systemu (Perspektywa Krystiana)

W trakcie sześćoetapowego cyklu deweloperskiego warstwa logiczna i architektoniczna ewoluowała od surowego szkieletu systemowego do w pełni zoptymalizowanego, deterministycznego silnika czasu rzeczywistego.

- Faza Inicjalizacji (Tygodnie 1–2): Skonstruowano stabilne środowisko uruchomieniowe w języku C++ z integracją biblioteki SDL. Kluczowym osiągnięciem było wdrożenie niskopoziomowej pętli gry (*Game Loop*) oraz modułów detekcji kolizji kursora z obiektami aktywnymi (*picking mechanics*).

- Faza Implementacji Logiki Ekonomicznej (Tygodnie 3–4): Zaprojektowano i zaimplementowano algorytmy skalowania progresywnego. Wdrożono maszynę stanów kontrolującą procesy transakcyjne oraz asynchroniczny podsystem timerów, umożliwiający pasywną generację zasobów w interwałach sekwencyjnych (1000 ms).
- Faza Konsolidacji i Optymalizacji (Tygodnie 5–6): Przeprowadzono refaktoryzację struktury kodu źródłowego w celu redukcji długu technicznego. Dokonano profilowania aplikacji pod kątem wycieków pamięci oraz zoptymalizowano algorytmy obliczeniowe, co zaowocowało maksymalną stabilizacją klatek na sekundę (FPS).

2. Warstwa Prezentacji i Persystencji Danych (Perspektywa Tymoteusza)

Prace nad interfejsem użytkownika (UI/UX) oraz strukturą składowania danych zapewniły spójność wizualną systemu oraz pełną integralność informacji w stanie rozproszonym.

- Faza Projektowania Struktur i Prototypowania (Tygodnie 1–2): Zainicjalizowano bezserwerowy system zarządzania bazą danych SQLite. Opracowano relacyjny model danych obejmujący tabele `player`, `upgrades` oraz `stats`. Równolegle wdrożono minimalistyczny interfejs graficzny dedykowany do obsługi zdarzeń wejściowych.
- Faza Integracji Transakcyjnej i Telemetrii (Tygodnie 3–4): Rozbudowano bazę danych o relacje dla systemów ulepszeń oraz zaimplementowano procedury zapisu transakcji w czasie rzeczywistym. Do interfejsu wprowadzono moduły telemetryczne, w tym dynamiczny wskaźnik wydajności sekundowej (PPS).
- Faza Estetyzacji, Nawigacji i Walidacji (Tygodnie 5–6): Testowe komponenty GUI zastąpiono docelowymi zasobami dwuwymiarowymi oraz zaimplementowano dynamiczne animacje sprzężenia zwrotnego (*Click Animation*). Sfinalizowano przepływ ekranów poprzez implementację Menu Startowego oraz Panelu Statystyk Historycznych, kończąc cykl pełną walidacją regresyjną systemu.

3. Wnioski Końcowe i Ewaluacja Rezultatów

1. Zgodność ze specyfikacją: Projekt został zrealizowany w 100% zgodnie z pierwotnymi założeniami funkcjonalnymi. Wszystkie zaplanowane kamienie milowe zostały osiągnięte w wyznaczonych interwałach czasowych.
2. Integralność technologiczna: Połączenie technologii C++ (wysoka wydajność obliczeniowa), SDL (bezpośredni dostęp do warstwy graficznej sprzętu) oraz SQLite (lekka, niezawodna persystencja danych) okazało się optymalnym wyborem architektonicznym dla aplikacji typu *incremental*.
3. Wartość użytkowa (UX): Zastosowanie asynchronicznego zapisu, dynamicznych animacji oraz przejrzystych paneli informacyjnych (statystyki, telemetria PPS) pozwoliło na uzyskanie wysokiego wskaźnika responsywności i ergonomii systemu.

Status Projektu: **ZAKOŃCZONY**